

**Факультет Программной инженерии и компьютерной техники
Образовательная программа Проектирование и разработка систем больших данных
Направление подготовки (специальность) 09.04.04 - Программная инженерия**

ОТЧЕТ

по лабораторной работе №1
по курсу «Структуры и алгоритмы в базах данных и распределенных системах»
на тему: «Реализация алгоритмов хэширования»

Обучающийся: Никифорова Е. А., Р4135

Преподаватель: Платонов А. В., доцент

Преподаватель: Портнов П. В., ассистент

Содержание

Задание	3
Реализация	4
Расширяемое хэширование (Extendible Hashing)	4
Структура данных в памяти и на диске	4
Основные структуры	4
Интерфейс	4
Механика работы	4
Идеальное хэширование (Perfect Hashing)	6
Структура данных в памяти	6
Основные структуры	6
Интерфейс	6
MinHash LSH	7
Алгоритм:	7
Интерфейс	7
Тестирование	9
Расширяемое хэширование (Extendible Hashing)	9
Результаты бенчмарков:	9
Результаты профайлинга perf:	10
Идеальное хэширование (Perfect Hashing)	13
Результаты бенчмарков:	13
MinHash LSH	14
Результаты бенчмарков:	14

Задание

Необходимо реализовать три алгоритма (в скобках операции):

1. Extendible хэш таблицу на файловой системе с бакетами(вставка, обновление и удаление). Прямая работа с диском и mmap
2. Perfect hash (создание полного индекса по фиксированному набору ключей и поиск)
3. LSH для поиска дублей в текстах или точек в 3d (создание индекса, добавление новых точек, фулл скан для поиска дублей)

По всем алгоритмам должны быть:

1. Тесты функциональности на случайных наборах данных (набор данных при тесте генерируется)
2. Перфтесты на больших объемах данных - считаем пропускную способность и время операции
3. Профайлинг памяти и цпу всех реализаций с анализом узких мест и набором гипотез как можно сделать лучше

Реализация

Расширяемое хэширование (Extendible Hashing)

Структура данных в памяти и на диске

Файл БД организован как непрерывный регион, разделенный на три секции:

1. **Header**: Хранит глобальные параметры (глубины, смещения, размеры).
2. **Directory**: Массив 64-битных смещений. Каждый элемент — это `bucket_offset` (смещение от начала файла до конкретного бакета).
3. **Buckets**: Блоки фиксированного размера. Каждый бакет содержит `BucketHeader` (локальная глубина и счетчик) и массив `Entry`.
4. **Entry**: Единица хранения - пара ключ-значение

Основные структуры

```
struct Header {
    uint32_t global_depth;           // Глобальная глубина
    uint32_t max_global_depth;      // Ограничение максимальной глубины
    uint64_t bucket_capacity;       // Сколько структур Entry влезает в один
    бакет.                          // Задается при создании таблицы.
    uint64_t bucket_count;          // Общее кол-во физических бакетов в
    файле.
    uint64_t first_bucket_offset;    // Смещение до начала бакетов
    uint64_t file_size;             // Общий размер файла
};

struct BucketHeader {
    uint32_t local_depth; // Локальная глубина
                        // Показывает, сколько бит хеша общие
                        // для всех ключей в этом бакете
    uint32_t count;
};

struct Entry {
    int64_t key;
    int64_t value;
};
```

Интерфейс

- `openTable / createTable`: Открытие существующего файла или создание нового с инициализацией метаданных.
- `insertRecord`: Добавление новой пары (K, V) . Если ключ существует — ошибка, если бакет полон — инициирует процедуру расщепления (`Split`).
- `getRecord`: Поиск значения по ключу
- `removeRecord / updateRecord`: Удаление или обновление значения существующего ключа.

Механика работы

- Используется системный вызов `mmap` для отображения файла базы данных напрямую в виртуальное адресное пространство процесса.
- Для любого ключа вычисляется индекс в директории через битовую маску:

- $\text{index} = \text{hash}(\text{key})((1 \ll \text{global_depth}) - 1)$
- Из директории извлекается смещение, которое складывается с базовым адресом для получения физического адреса бакета.
- Переполнение (Split)
 - Если $\text{local_depth} < \text{global_depth}$:
 - создается новый бакет, элементы перераспределяются, часть указателей директории перекидывается на новый адрес.
 - Если $\text{local_depth} == \text{global_depth}$:
 - сначала удваивается директория, $\text{global_depth}++$, затем стандартный Split.
- Расширение (Remapping):
 - При создании бакета файл физически растет через `ftruncate`.
 - Старый `mmap` сбрасывается, и создается новый на увеличенный размер.

Идеальное хэширование (Perfect Hashing)

Структура данных в памяти

- Perfect Hashing (FKS) строится один раз для статического набора ключей, обеспечивая поиск за $O(1)$ в худшем случае.
- Первый уровень (Глобальный): Хранит параметры хеш-функции и массив указателей на подтаблицы.
- Второй уровень (Локальный): Набор независимых подтаблиц (SubTable).
 - Если в бакет попало n_i элементов, размер подтаблицы устанавливается как n_i^2 .

Основные структуры

```
++
struct Entry {
    int64_t key;
    int64_t value;
    static constexpr int64_t kEmpty = std::numeric_limits<int64_t>::min();
};

struct HashParams {
    uint64_t a; // Случайный множитель
    uint64_t b; // Случайный сдвиг
    uint64_t m; // Размер таблицы
};

struct SubTable {
    HashParams params;           // Локальные параметры для 2-го уровня
    std::vector<Entry> cells;    // Массив пар Key-Value
    bool initialized;           // Флаг наличия данных в корзине
};

struct PerfectHash {
    size_t _n;                  // // Общее кол-во уникальных
    // ключей
    HashParams _first_level_params; // // Параметры основной функции
    std::vector<SubTable> _second_level_tables;
};
```

Интерфейс

- PerfectHash(keys): Конструктор. Выполняет предварительную фильтрацию дубликатов,

распределение по корзинам и итеративный подбор параметров a, b для каждого уровня до тех пор, пока не будет достигнуто отсутствие коллизий.

- get(key): Поиск. Возвращает std::optional.

Выполняет ровно два вычисления хеша и одну проверку ключа в ячейке.

MinHash LSH

Реализация алгоритма MinHash LSH, предназначенная для эффективного поиска похожих текстовых документов в большом наборе данных. Основная идея заключается в том, чтобы не сравнивать каждый документ с каждым, а быстро находить **кандидатов** на сходство, а затем уже проверять их более детально.

Алгоритм:

1. **Извлечение шинглов (Shingling):** Текст разбивается на небольшие перекрывающиеся последовательности слов (шинглы).
2. **MinHashing:** Для каждого документа создается «сигнатура» — короткий вектор чисел, который статистически сохраняет информацию о сходстве между наборами шинглов документов. Сходство сигнатур приблизительно соответствует сходству исходных документов.
3. **LSH:** Сигнатуры документов делятся на полосы. Каждая полоса хешируется в корзину в отдельной хеш-таблице. Если две сигнатуры имеют хотя бы одну полосу, попадающую в одну и ту же корзину, они считаются кандидатами на сходство. Это позволяет значительно сузить круг потенциально похожих документов.
4. **Проверка сходства:** Для найденных кандидатов вычисляется коэффициент Жаккара между их исходными наборами шинглов, чтобы подтвердить или опровергнуть их дублирование.

Интерфейс

- `MinHashLSH(m_permutations, b_bands, shingle_size)`: Конструктор. Инициализирует систему MinHash LSH, задавая основные параметры:
 - `m_permutations`: Количество перестановок(хеш-функций) для построения сигнатуры.
 - `b_bands`: Количество полос, на которые делится сигнатура для LSH.
 - `shingle_size`: Размер шингла (количество слов в одном фрагменте текста).Генерирует уникальный набор перестановочных хеш-функций.
- `addDocument(text)`: Добавляет новый документ в систему. Выполняет нормализацию текста, извлечение шинглов, построение MinHash-сигнатуры и распределение её по соответствующим корзинам LSH-таблиц. Возвращает `DocID` добавленного документа.
- `findCandidates(text)`: Находит потенциальных кандидатов на сходство для заданного нового текста. Вычисляет MinHash-сигнатуру для `text` и ищет в LSH-таблицах документы, попавшие в те же корзины, что и сигнатура входного текста. Возвращает `std::set<DocID>` — набор идентификаторов документов-кандидатов.
- `findCandidatesById(doc_id)`: Находит потенциальных кандидатов на сходство для уже существующего в системе документа по его `DocID`. Использует ранее сохраненную MinHash-сигнатуру документа `doc_id` для поиска в LSH-таблицах. Возвращает `std::set<DocID>` — набор идентификаторов документов-кандидатов (исключая сам `doc_id`).
- `findDuplicatesFullScan(text, threshold)`: Выполняет полную проверку на дубликаты для заданного текста. Сравнивает `text` со всеми ранее добавленными документами, вычисляя точный коэффициент Жаккара между их наборами шинглов. Если коэффициент Жаккара превышает `threshold` (заданный порог сходства),

документ считается дубликатом. Возвращает `std::vector<DocID>` — список идентификаторов документов, признанных дубликатами.

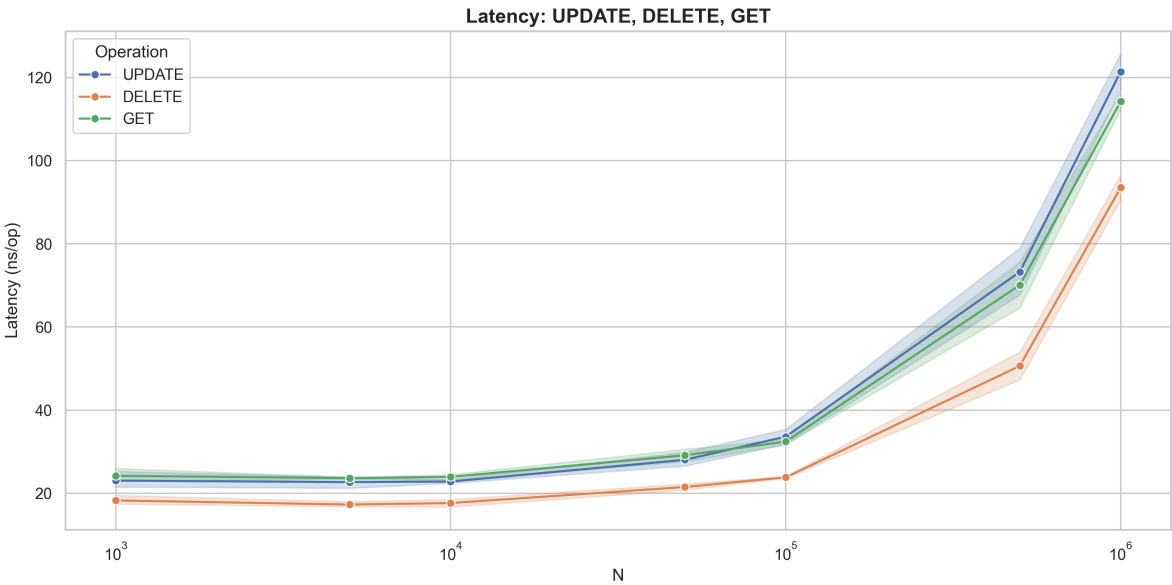
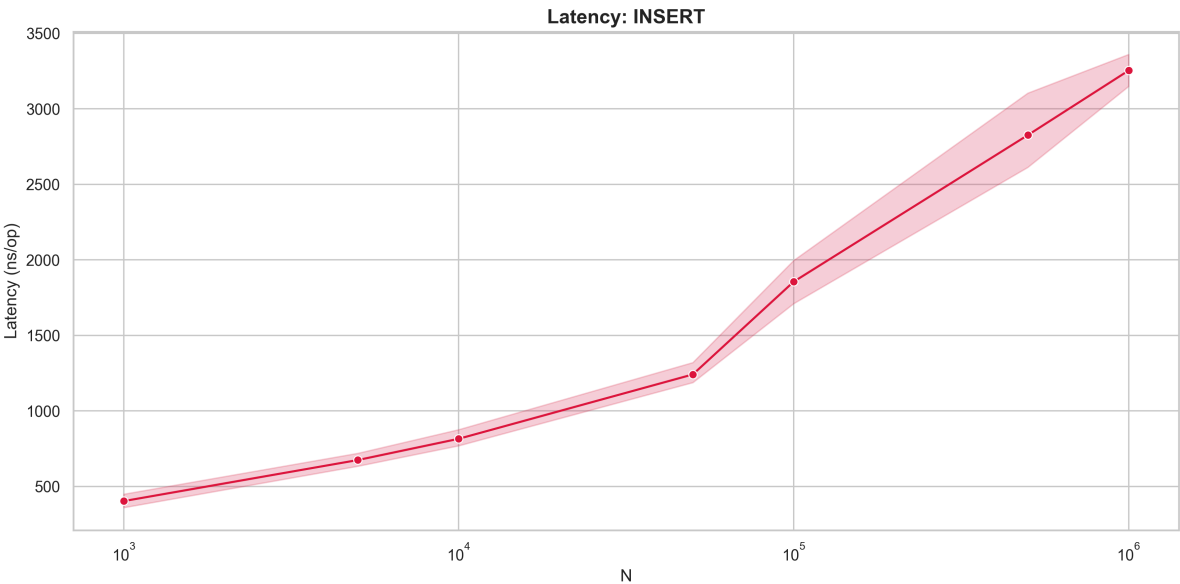
Тестирование

Расширяемое хэширование (Extendible Hashing)

Результаты бенчмарков:

=== РЕЗУЛЬТАТЫ ЗАДЕРЖКИ (среднее ± 95% CI, ns/op) ===

N	INSERT	UPDATE	DELETE	GET
1000	402.86 ± 48.95	23.04 ± 2.22	18.27 ± 1.22	24.21 ± 1.69
5000	674.72 ± 47.86	22.69 ± 1.51	17.29 ± 0.67	23.64 ± 0.26
10000	815.33 ± 60.18	22.82 ± 0.53	17.65 ± 0.94	23.94 ± 0.88
50000	1241.41 ± 80.32	28.05 ± 1.78	21.51 ± 0.84	29.14 ± 1.84
100000	1855.40 ± 174.07	33.61 ± 2.05	23.84 ± 0.28	32.42 ± 0.75
500000	2826.00 ± 296.22	73.23 ± 6.68	50.62 ± 3.73	70.08 ± 6.51
1000000	3253.71 ± 121.65	121.39 ± 4.83	93.49 ± 3.36	114.21 ± 2.19



- ```

=== Сбор метрик для малого N (10,000) ===

Performance counter stats for './../cmake-build-relwithdebinfo/lab1/extendible_bench_tests 10000':

 1650457 cache-misses
 59131 LLC-load-misses

 0.205339067 seconds time elapsed

 0.028473000 seconds user
 0.119996000 seconds sys

=== Сбор метрик для большого N (1,000,000) ===

Performance counter stats for './../cmake-build-relwithdebinfo/lab1/extendible_bench_tests 1000000':

 403956478 cache-misses
 112157744 LLC-load-misses

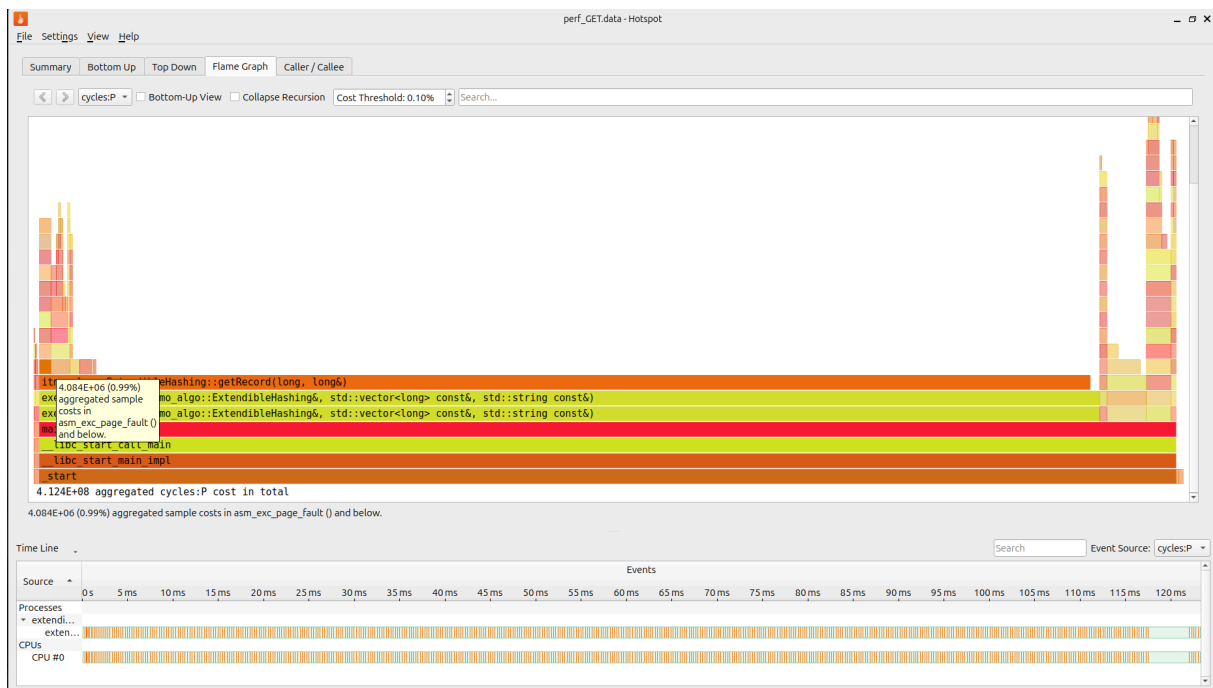
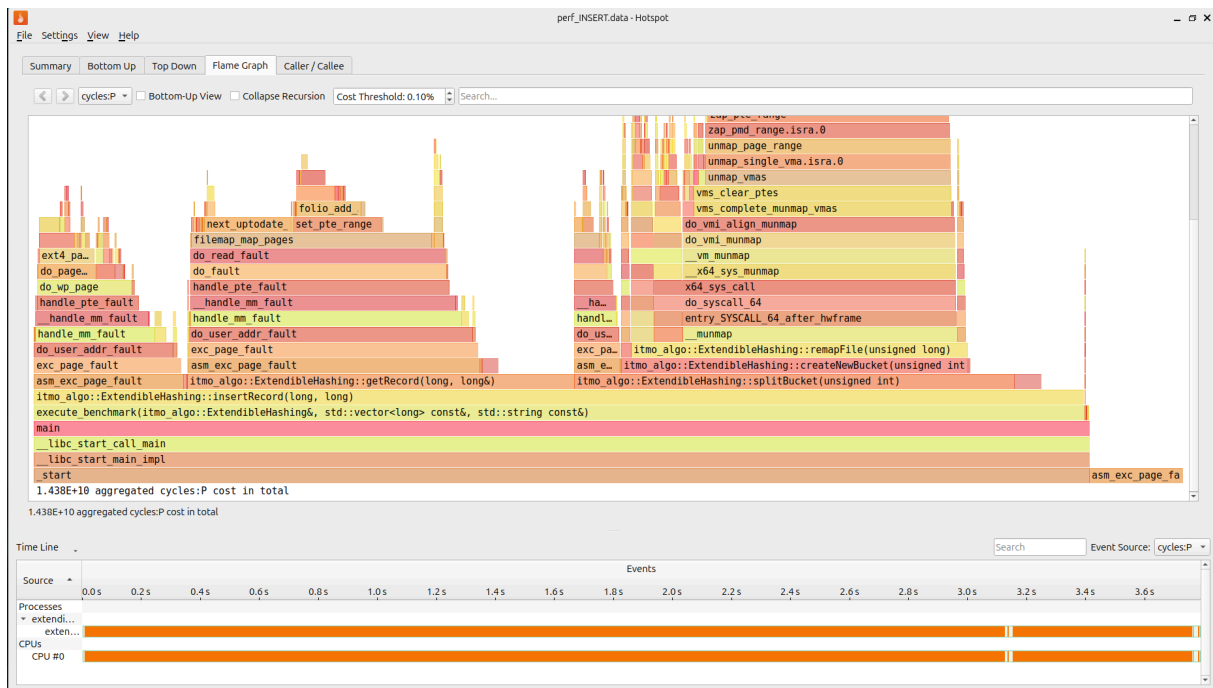
 17.585242454 seconds time elapsed

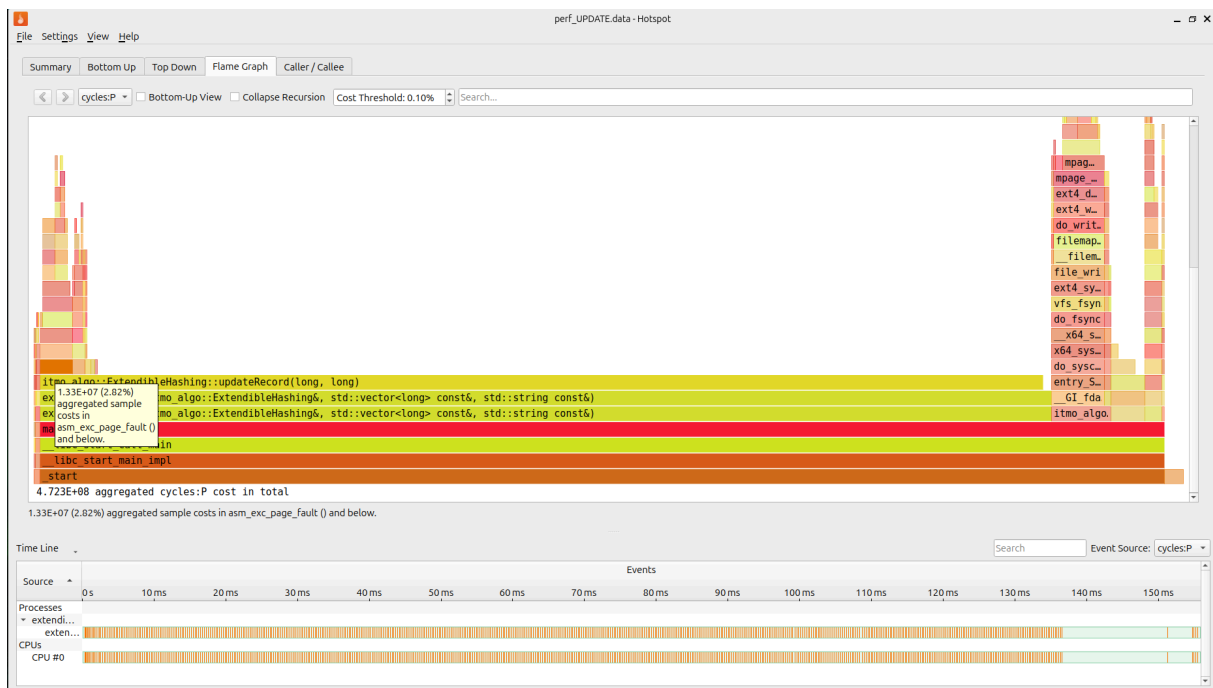
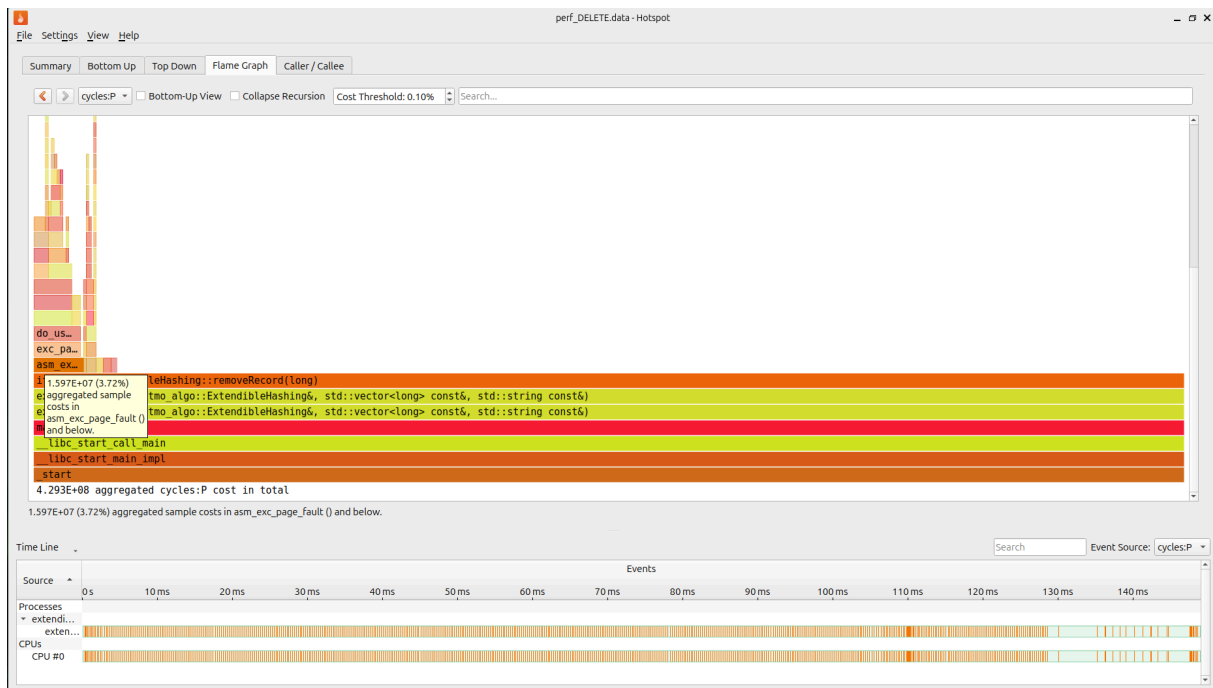
 4.085343000 seconds user
 13.190930000 seconds sys

```

[illegible]

- Основное время тратится в функциях ядра «mmap». Программа простаивает на системных вызовах при изменении размера файла.
- Высокая нагрузка в `do_page_fault`. Процессор ждет, пока ОС выделит физическую память под «ленивый» `mmap`.





## Идеальное хэширование (Perfect Hashing)

Результаты бенчмарков:

| === РЕЗУЛЬТАТЫ ЗАДЕРЖКИ (среднее, ns/op) ===                  |       |        |        |
|---------------------------------------------------------------|-------|--------|--------|
| N                                                             | BUILD | GET    |        |
| -----                                                         |       |        |        |
| 1000                                                          |       | 156.03 | 14.22  |
| 5000                                                          |       | 235.64 | 25.94  |
| 10000                                                         |       | 221.88 | 56.28  |
| 50000                                                         |       | 230.95 | 84.90  |
| 100000                                                        |       | 315.06 | 113.90 |
| 500000                                                        |       | 488.66 | 133.60 |
| 1000000                                                       |       | 455.54 | 108.87 |
|                                                               |       |        |        |
| === РЕЗУЛЬТАТЫ ПРОПУСКНОЙ СПОСОБНОСТИ (среднее, млн оп/с) === |       |        |        |
| N                                                             | BUILD | GET    |        |
| -----                                                         |       |        |        |
| 1000                                                          |       | 6.41   | 70.31  |
| 5000                                                          |       | 4.24   | 38.55  |
| 10000                                                         |       | 4.51   | 17.77  |
| 50000                                                         |       | 4.33   | 11.78  |
| 100000                                                        |       | 3.17   | 8.78   |
| 500000                                                        |       | 2.05   | 7.49   |
| 1000000                                                       |       | 2.20   | 9.19   |

С увеличением числа элементов построение таблицы (BUILD) становится значительно медленнее и менее производительным. При этом операции получения (GET), хоть и показывают некоторое замедление, остаются весьма эффективными. Основная вычислительная нагрузка при использовании идеального хеширования приходится на этап построения самой таблицы.

# MinHash LSH

## Результаты бенчмарков:

=== ЗАДЕРЖКА (среднее, ms/op) ===

| Docs (N) |  | ADD    |  | LSH FIND |  | FULL SCAN FIND |
|----------|--|--------|--|----------|--|----------------|
| -----    |  |        |  |          |  |                |
| 100      |  | 0.0963 |  | 0.0925   |  | 0.4444         |
| 500      |  | 0.0912 |  | 0.0890   |  | 1.9318         |
| 1000     |  | 0.0727 |  | 0.0736   |  | 4.3528         |
| 5000     |  | 0.0976 |  | 0.1115   |  | 21.8066        |
| 10000    |  | 0.1036 |  | 0.1124   |  | 50.3468        |
| 20000    |  | 0.0959 |  | 0.1423   |  | 95.5152        |

=== ПРОПУСКНАЯ СПОСОБНОСТЬ (оп/сек) ===

| Docs (N) |  | ADD        |  | LSH FIND   |  | FULL SCAN FIND |
|----------|--|------------|--|------------|--|----------------|
| -----    |  |            |  |            |  |                |
| 100      |  | 10382.4313 |  | 10813.1850 |  | 2250.1091      |
| 500      |  | 10963.3561 |  | 11234.0882 |  | 517.6517       |
| 1000     |  | 13756.1785 |  | 13581.6475 |  | 229.7348       |
| 5000     |  | 10250.1577 |  | 8967.7750  |  | 45.8577        |
| 10000    |  | 9649.3245  |  | 8898.1759  |  | 19.8622        |
| 20000    |  | 10427.1958 |  | 7027.6563  |  | 10.4695        |

- ADD: Показывает стабильное время выполнения вне зависимости от размера коллекции. Вычисления зависят от длины добавляемого текста. Время уходит на токенизацию, генерацию шинглов, вычисление MinHash-сигнатур и вставку их хешей
- FULL SCAN FIND: Демонстрирует предсказуемую линейную деградацию производительности. Алгоритм вынужден в лоб вычислять пересечение множеств, вычислять расстояние Жаккара
- LSH FIND: Алгоритм мгновенно извлекает только потенциально похожих кандидатов. Незначительный рост задержки при увеличении обусловлен повышением вероятности коллизий в бакетах: алгоритму приходится извлекать и объединять в `std::set` чуть большее количество ID кандидатов.